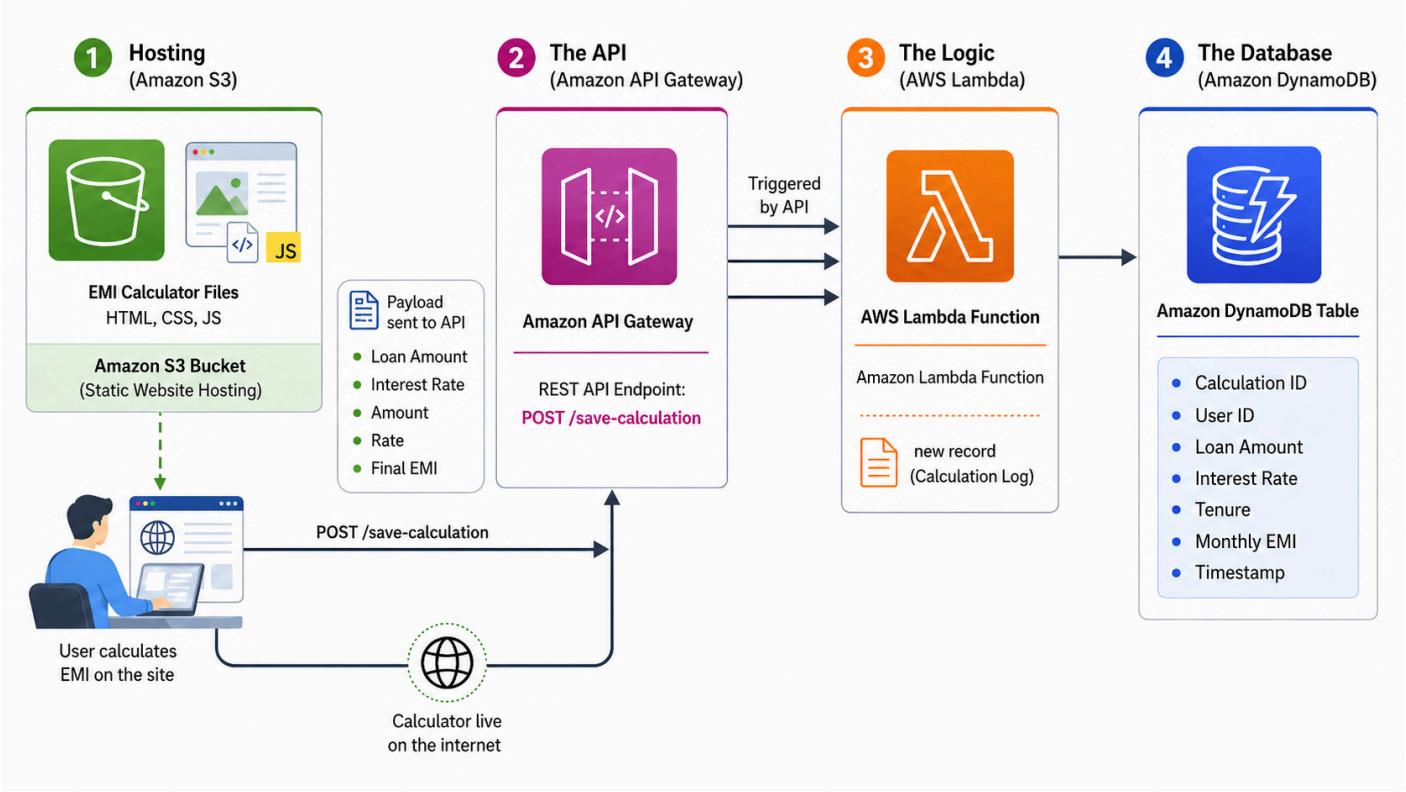
Cloud Hosted EMI Calculator: Serverless Architecture Migration

Objective: To migrate a static, client-side EMI calculator into a fully functional, serverless web application so as to persist user data or track calculation history.

Solution: Implemented an AWS serverless backend (API Gateway, Lambda, DynamoDB) to capture, process, and permanently store calculation records while maintaining a highly responsive, globally distributed frontend via Amazon S3.



Phase 1: The Database (Amazon DynamoDB)
The DynamoDB table stores the history of calculations on the EMI Calculator Web Application.

1) **Table details:**
    - Table name**:** EMI_History
    - Partition key: calculationId (Type: String). This will be a unique ID for every calculation.
2) **Table settings:** On-Demand capacity mode Under "Read/write capacity settings". This ensures payment as per usage, exactly $0.00 while the table sits idle.

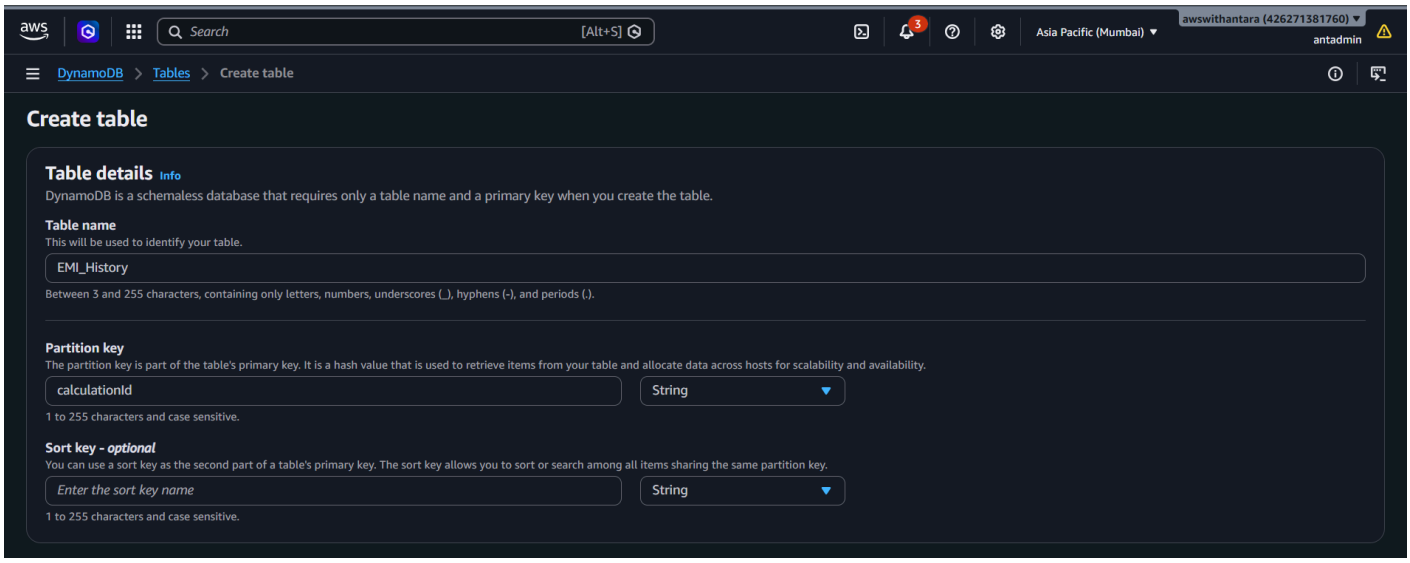


Table Creation in DynamoDB: Table Name & Partition Key

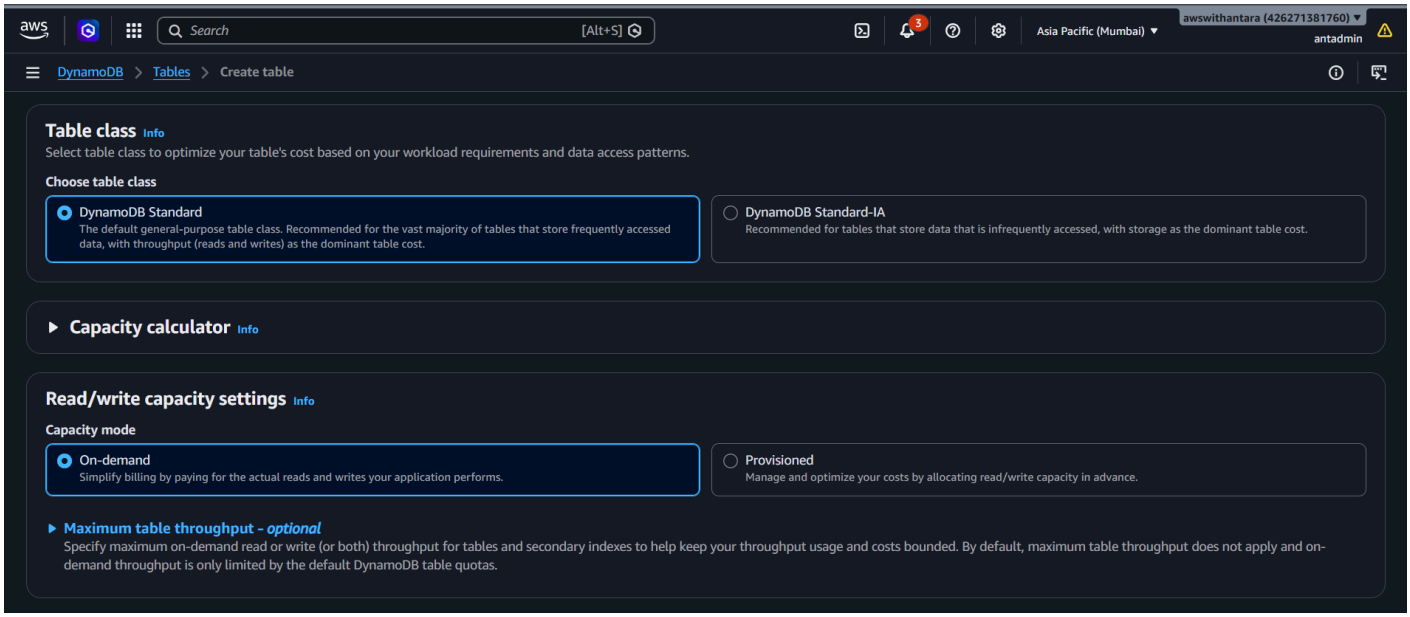


Table Creation in DynamoDB: Table Class & Read/Write Capacity Settings

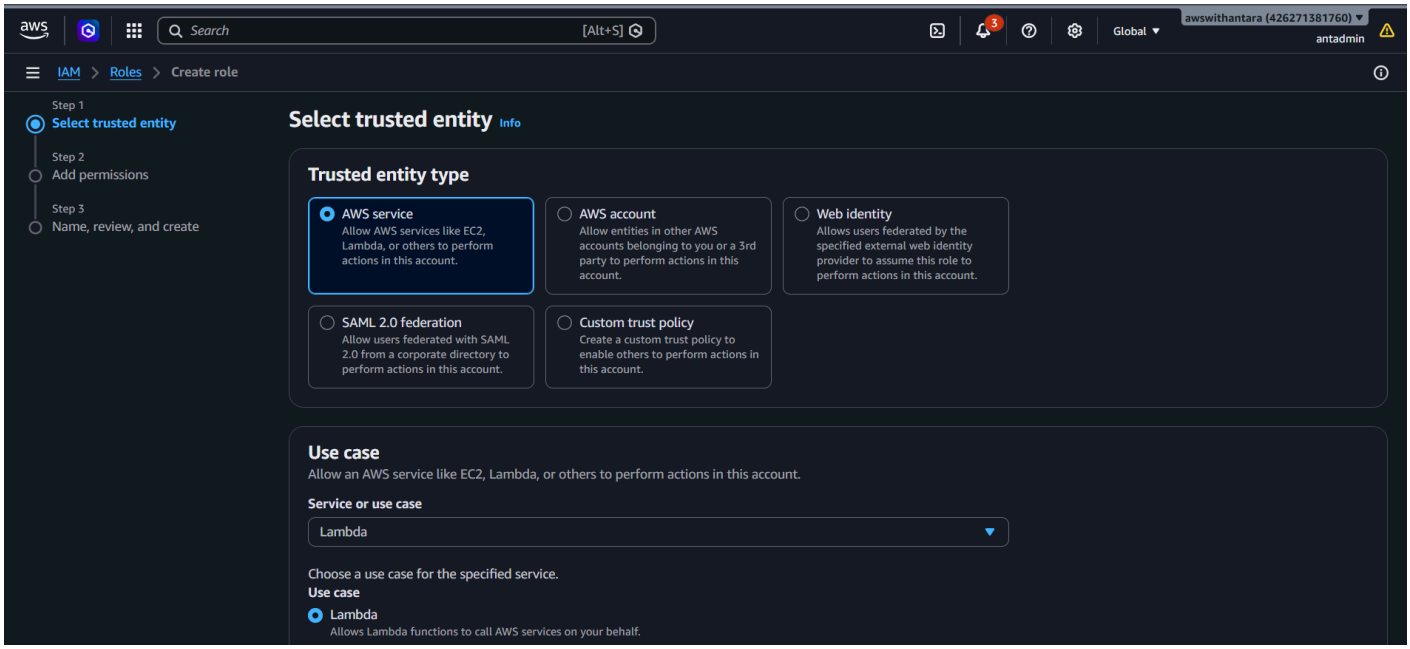
Phase 2: The Compute Logic (AWS Lambda & IAM)

The Lambda function receives data from the frontend (EMI Calculator Web Application) and saves it to the DynamoDB table (**EMI_History**). Also, an IAM Role is created for the Lambda function so that it can put data into the DynamoDB table.

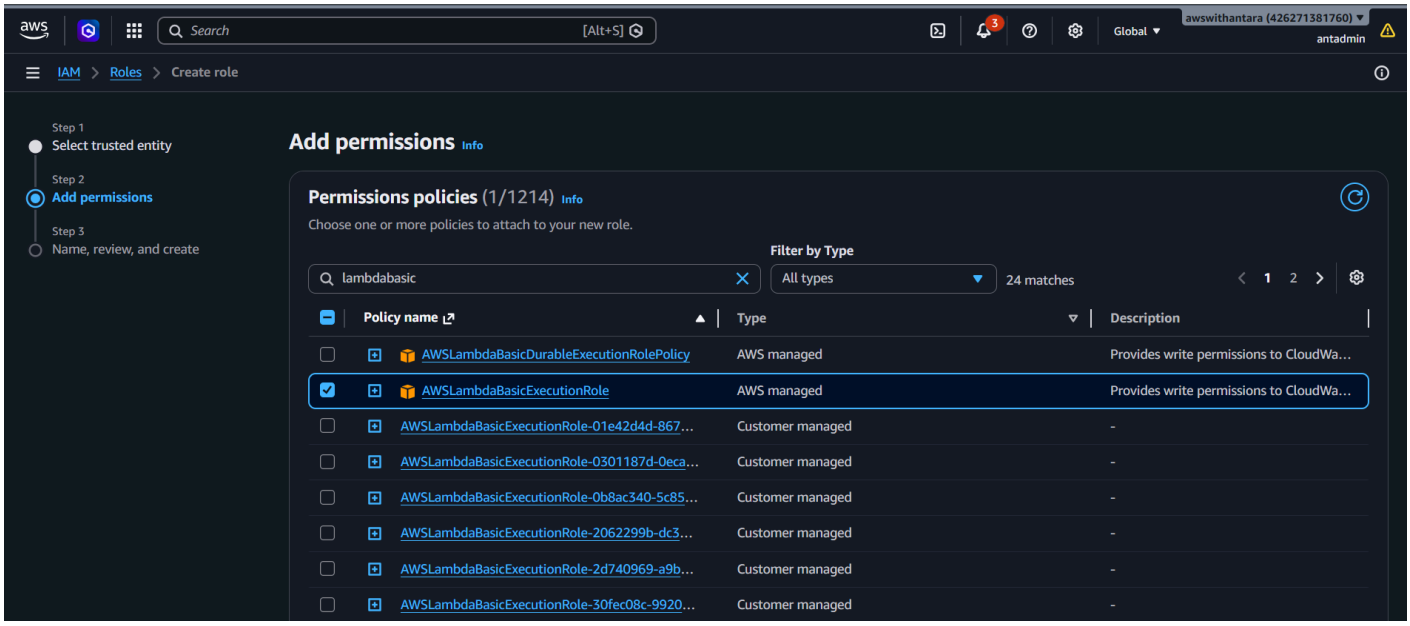
1) **IAM Role details:**

- Role name: EMICalculatorLambdaRole
- Entity type: AWS service
- Use case: Lambda

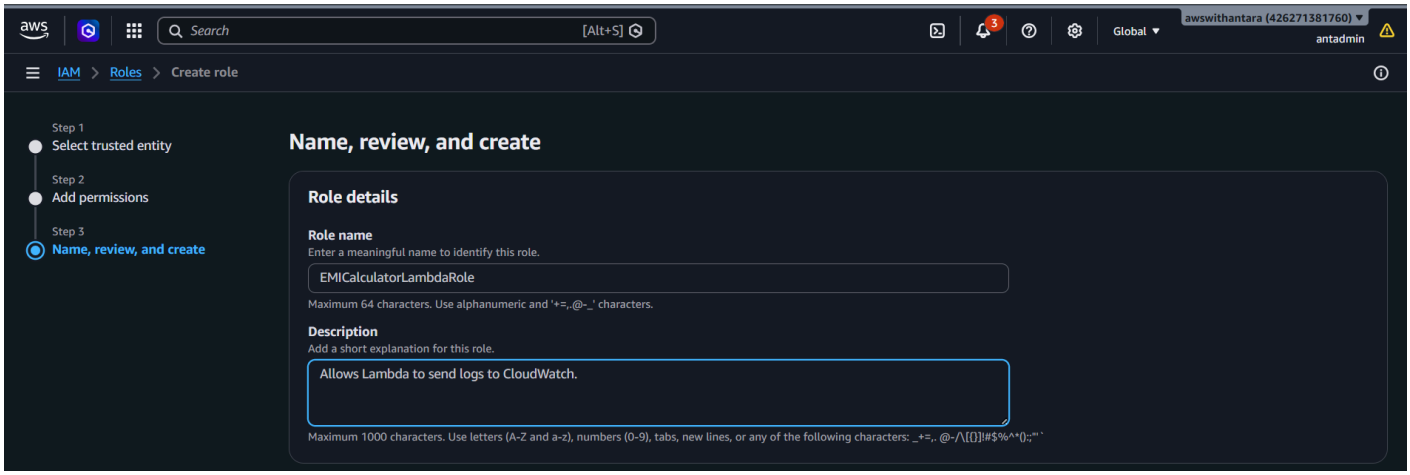
- Permissions policies: **AWSLambdaBasicExecutionRole** (so it can log to CloudWatch)
- Inline policy: An inline policy **PutItemDynamoDB** is created with dynamodb:PutItem on the **ARN** of the **EMI_History** table. It is attached to **EMICalculatorLambdaRole** which allows the Lambda function to put items in the DynamoDB table.



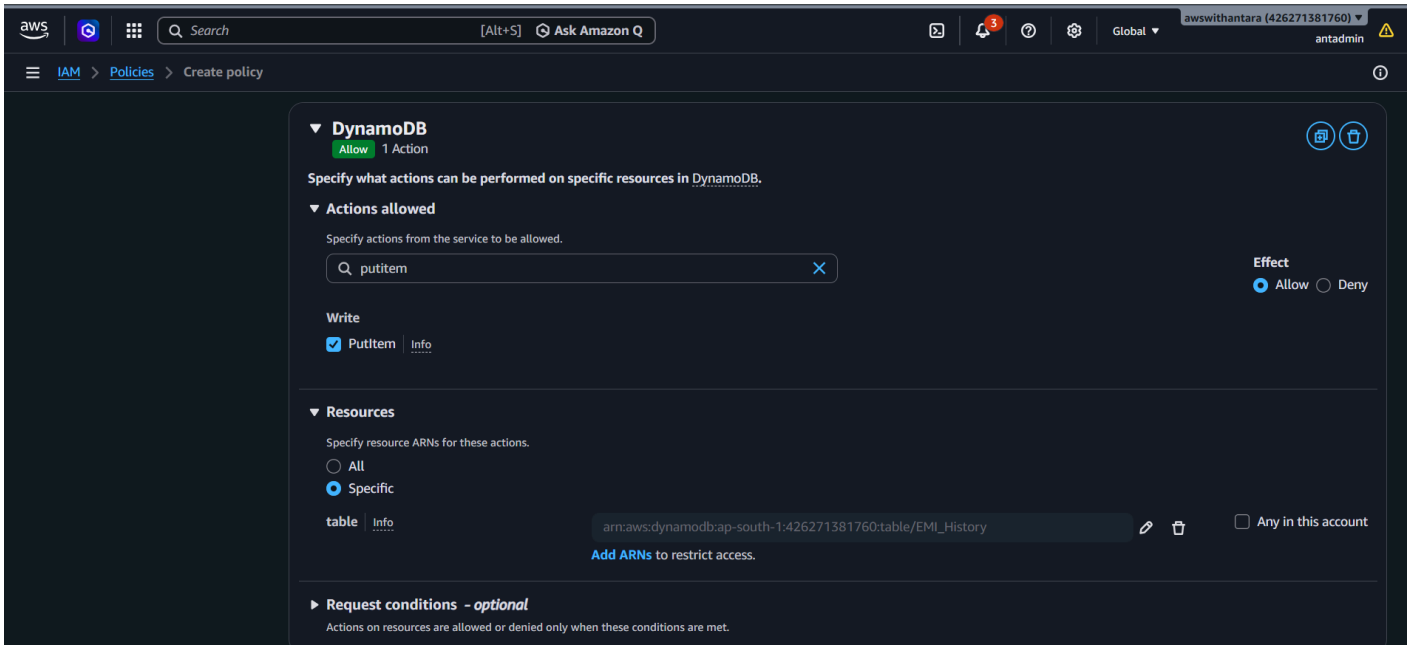
Role Creation in IAM: Entity Type & Use Case



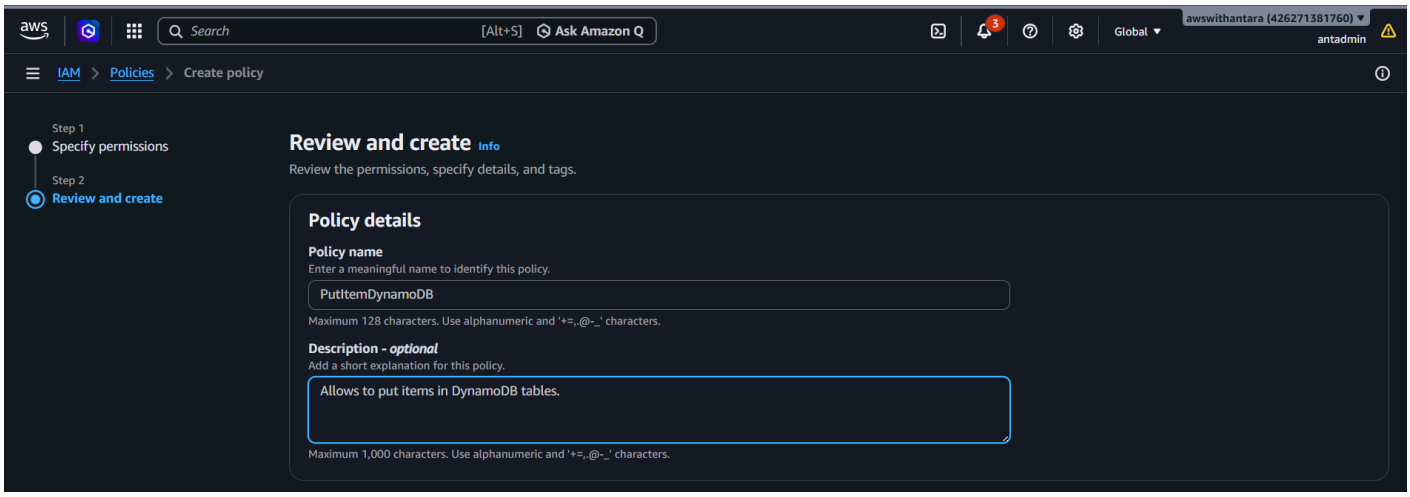
Role Creation in IAM: Permissions Policies



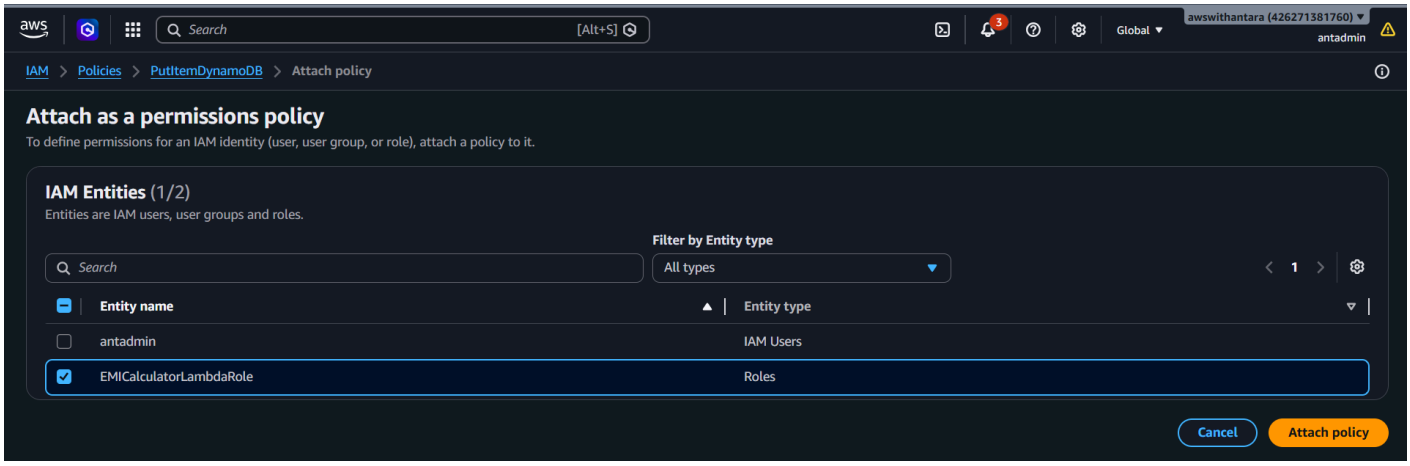
Role Creation in IAM: Role Name & Description



Policy Creation in IAM: Actions Allowed on DynamoDB with specific ARN



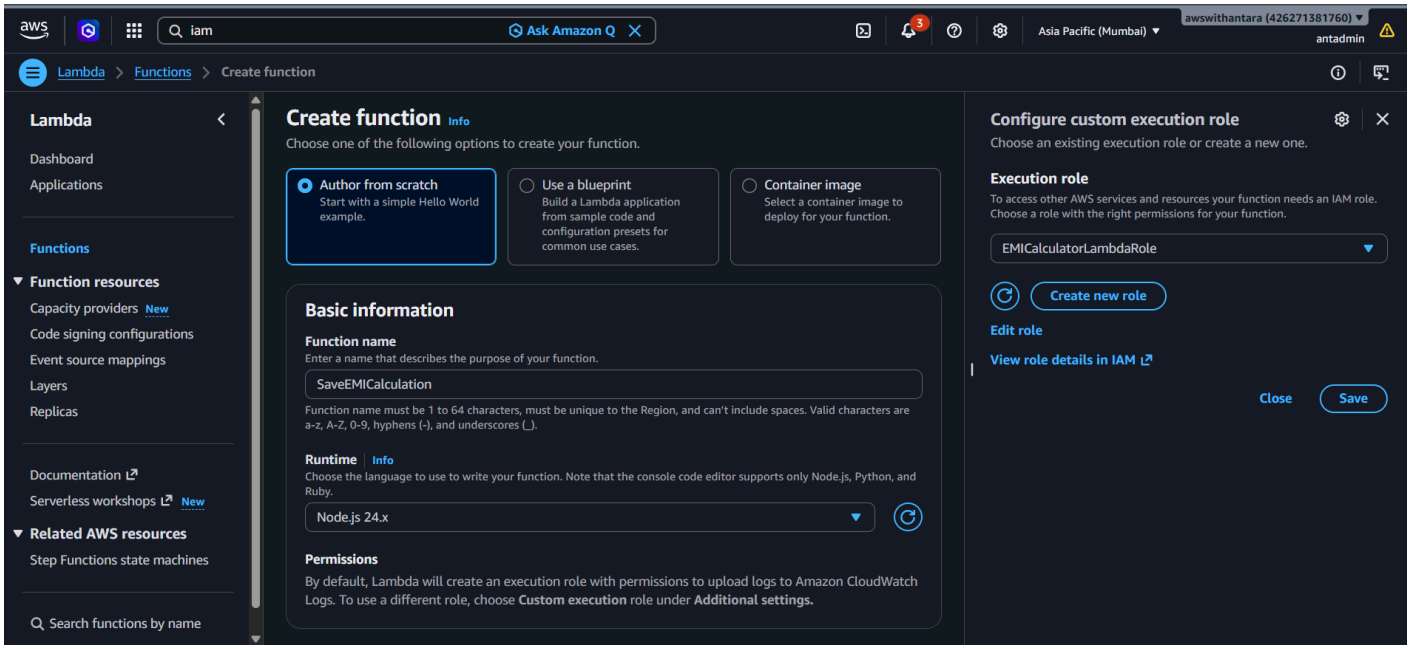
Policy Creation in IAM: Policy Name and Description



Edited IAM Policy: Attach the IAM Policy to the IAM Role

2) Lambda Function Details:

- Name: SaveEMICalculation
- Runtime: Nodejs 24.x
- Execution Role: Custom created execution role(IAM Role) **EMICalculatorLambdaRole**



Lambda Function Creation: Name, Runtime & Custom Execution Role

Lambda Code:

JavaScript

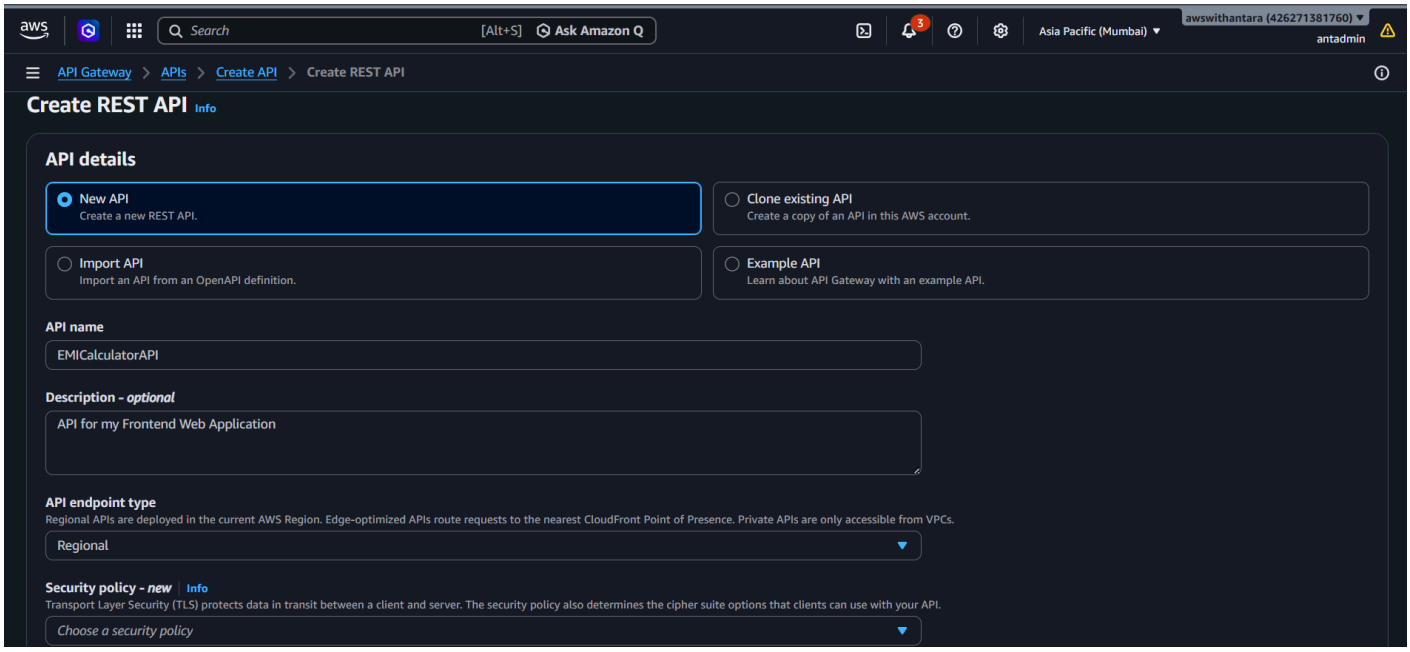
```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, PutCommand } from "@aws-sdk/lib-dynamodb";
import { randomUUID } from "crypto";

const client = new DynamoDBClient({});
const dynamo = DynamoDBDocumentClient.from(client);
const tableName = "EMI_History";

export const handler = async (event) => {
    // 1. Log the raw incoming request (Viewable in AWS CloudWatch)
    console.log("Incoming request body:", event.body);

    let body;
    try {
        // Parse the JSON string sent from your frontend fetch()
        body = JSON.parse(event.body);
    } catch (parseError) {
        return {
            statusCode: 400,
            headers: { "Access-Control-Allow-Origin": "*" },
            body: JSON.stringify({ error: "Invalid JSON payload format" })
        };
    }

    // 2. Map the incoming frontend payload to DynamoDB attributes
    const params = {
        TableName: tableName,
        Item: {
            calculationId: randomUUID(),
            timestamp: new Date().toISOString(),
            principalAmount: body.principal,
            interestRate: body.rate,
            tenureMonths: body.tenure,
            calculatedEMI: body.emi,
            totalInterestOutgo: body.totalInterest,
            totalOverallPayment: body.totalPayment
        }
```

```
    };

    // 3. Execute the database save
    try {
        await dynamo.send(new PutCommand(params));

        return {
            statusCode: 200,
            // CORS headers are strictly required here for your S3 hosted
site to accept the response
            headers: {
                "Access-Control-Allow-Origin": "*",
                "Access-Control-Allow-Methods": "OPTIONS,POST",
                "Access-Control-Allow-Headers": "Content-Type"
            },
            body: JSON.stringify({ message: "Calculation history saved to
AWS!" })
        };
    } catch (error) {
        console.error("DynamoDB Write Error:", error);

        return {
            statusCode: 500,
            headers: { "Access-Control-Allow-Origin": "*" },
            body: JSON.stringify({ error: "Failed to connect to the
database" })
        };
    }
};
```

Phase 3: The Bridge(API Gateway)
The frontend (EMI Calculator Web Application) cannot communicate with lambda directly. It needs an API endpoint.

1) **REST API Details:**

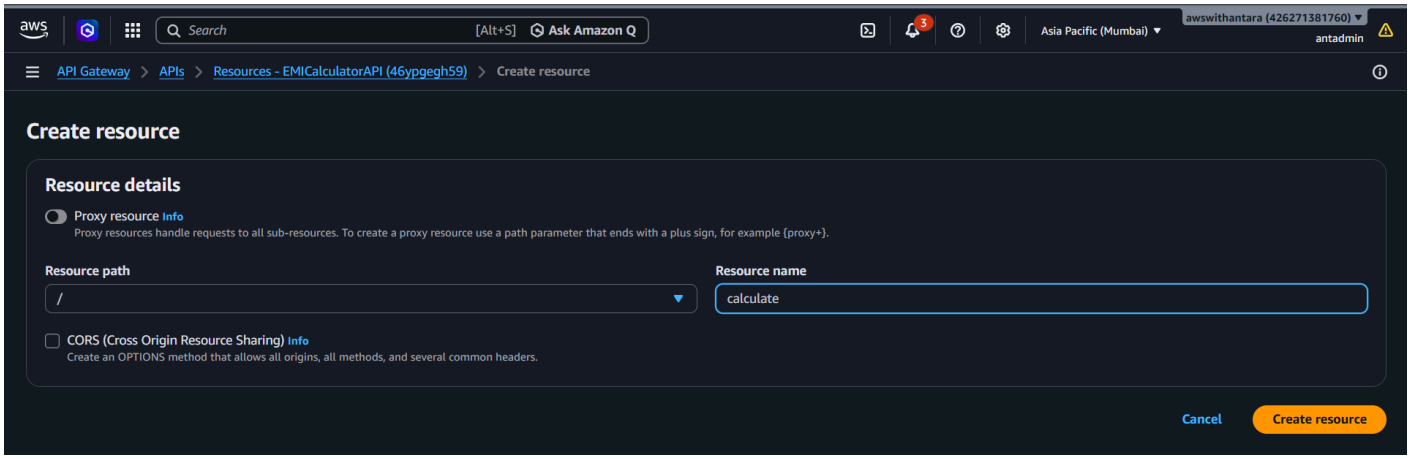
- Name: EMICalculatorAPI
- API endpoint type: Regional



API Gateway: REST API Creation

2) Resource Details:

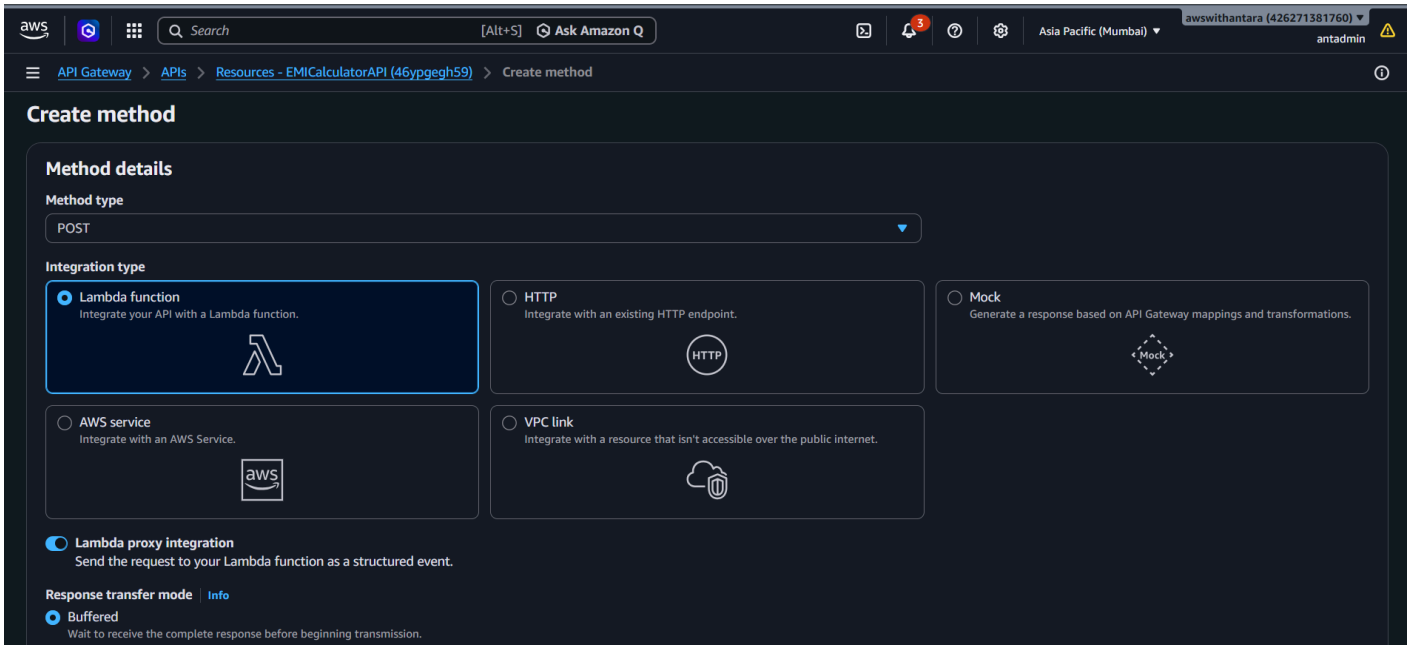
- Resource path: /
- Resource name: calculate



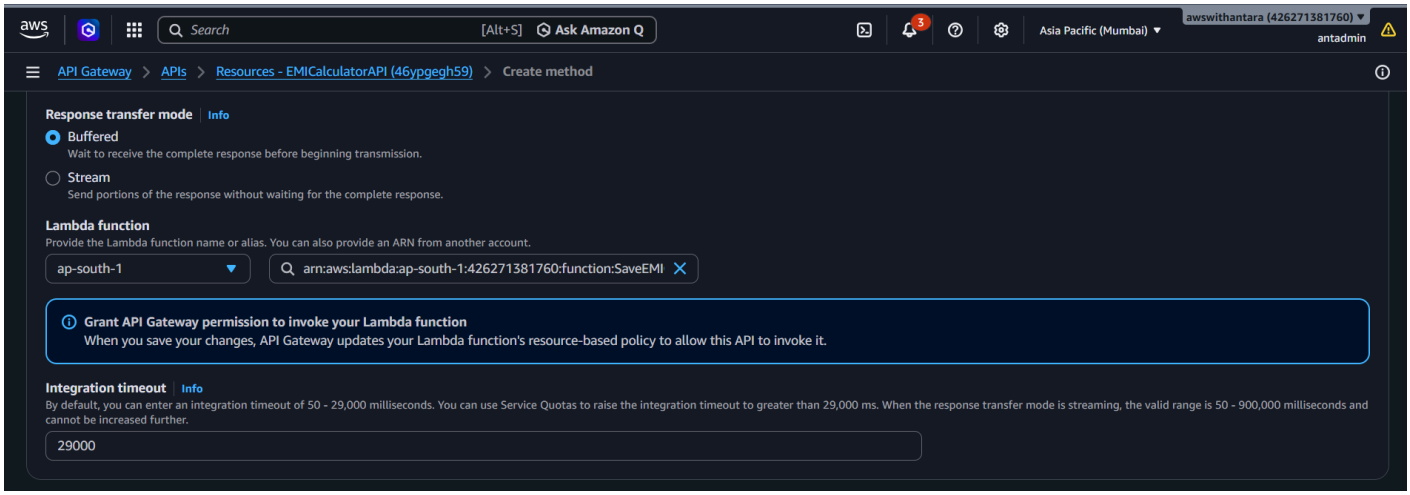
REST API Resource Creation: Resource Path & Name

3) Method Details:

- Method type: POST
- Integration type: Lambda Function
- Lambda proxy integration: Must be enabled, it passes the entire HTTP request to the Lambda code.



Method Creation for REST API Resource: Method & Integration type, Lambda Proxy

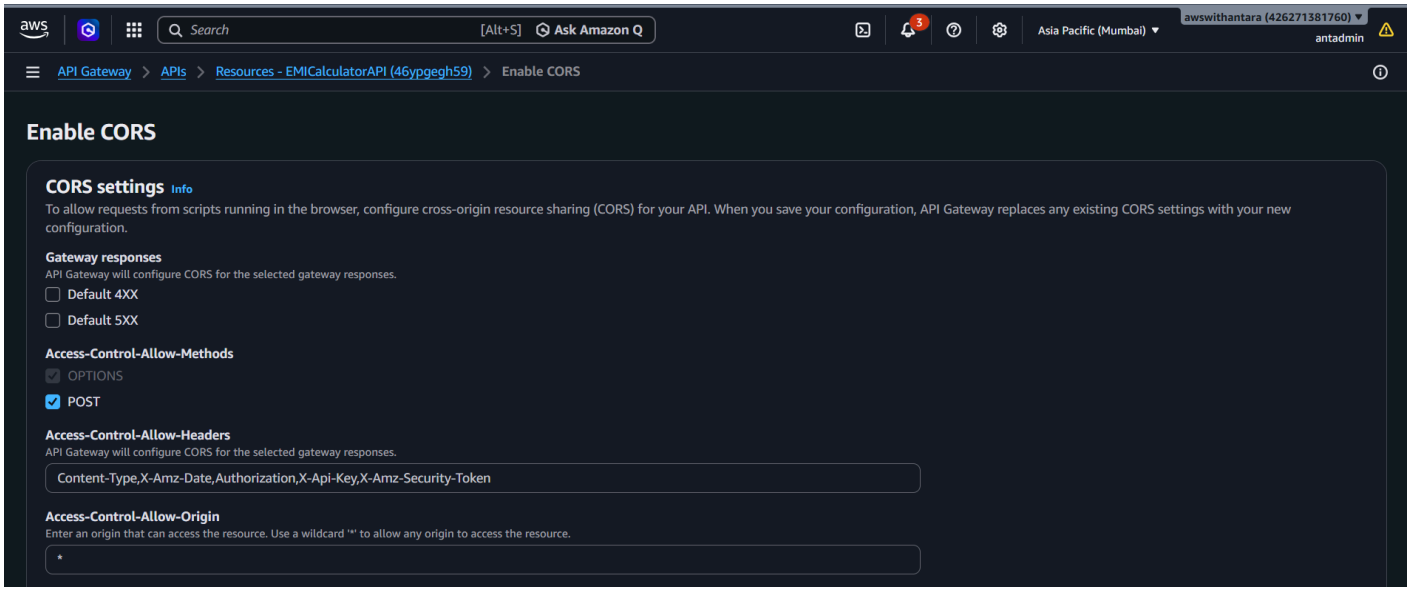


Method Creation for REST API Resource: Lambda Function Integration

4) CORS (Cross Origin Resource Sharing) Details:

- Access Control Allow Methods: OPTIONS & POST
- Access Control Allow Headers: Must include Content-Type so as to match frontend fetch request headers
- Access Control Allow Origin: * (any origin)

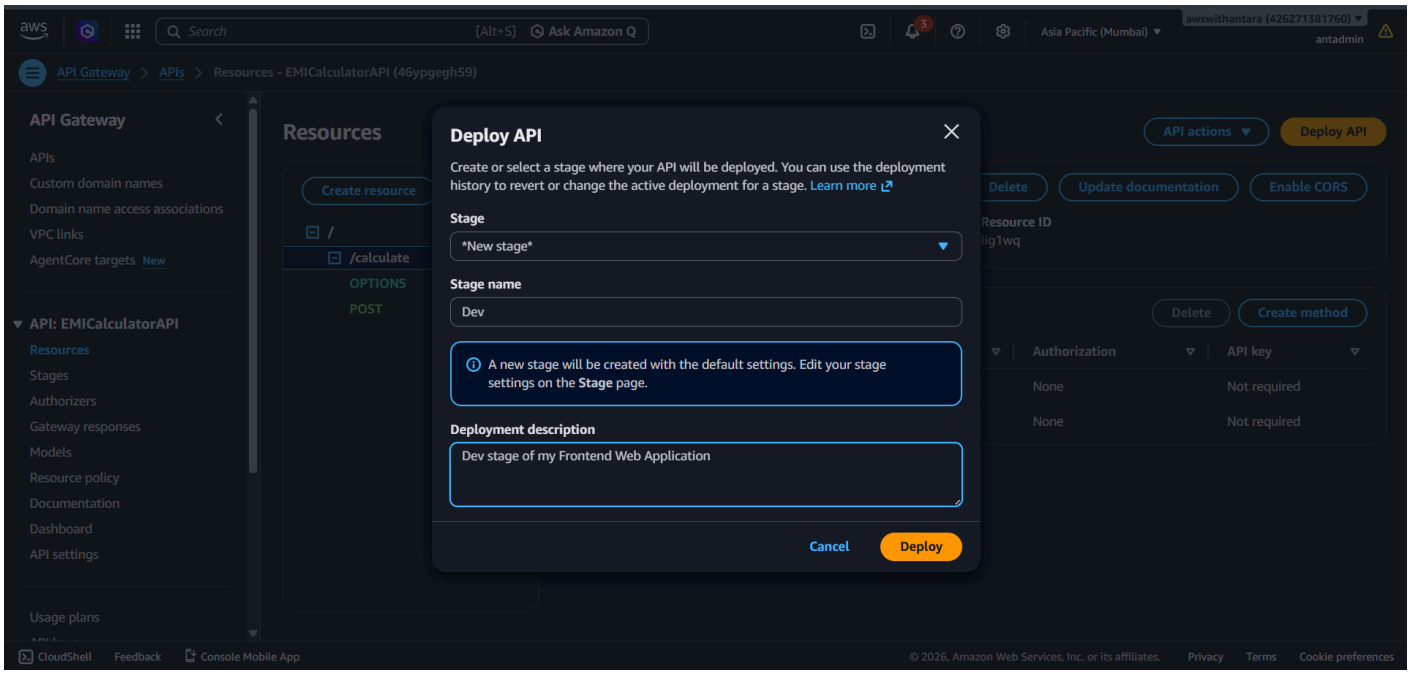
CORS is enabled and configured for the **/calculate** resource because such permissions are tied to specific endpoints. When the frontend JavaScript makes a fetch request to the API URL, the web browser first sends an invisible "preflight" request to that exact path which must explicitly grant permissions.



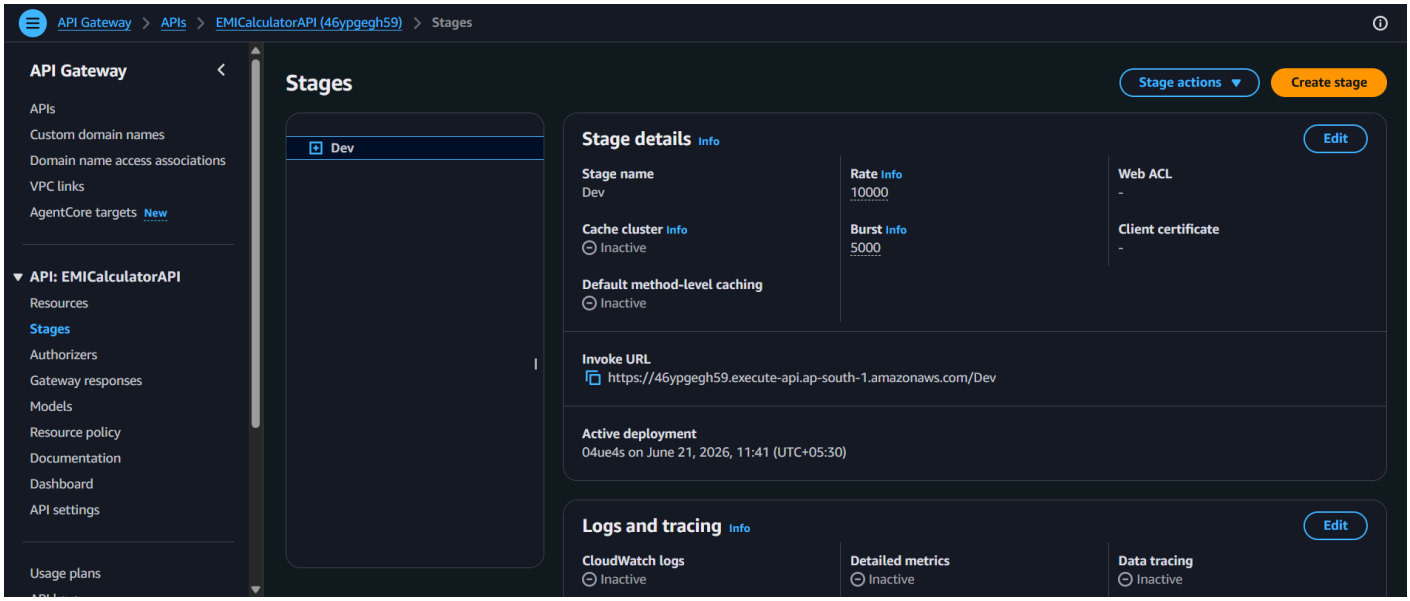
CORS Configuration for REST API Resource: Access Control Allow Settings

5) Deploy API:

- Stage: *New Stage*
- Stage Name: Dev
- Description: Dev stage of my frontend web application



REST API Deploy: Stage Creation



REST API Dev Stage: Stage Details

Phase 4: Frontend Integration & Request Optimization

The Frontend Web Application is connected to the API Gateway (**EMICalculatorAPI**) without causing rate-limiting or excessive AWS resource consumption from live interactions.

1) **Implementation Details:**

- Formatted a JSON payload inside **script.js** of the frontend application extracting the live state of Principal, Rate, Tenure, EMI, Total Interest, and Total Payment.
- Implemented an asynchronous **fetch()** POST request targeting the live API Gateway **/calculate** resource.

2) **Performance Optimization:**

- The frontend UI includes active sliders that fire rapid recalculations, hence a Debounce function is integrated (**setTimeout**)
- This strategy prevents API Gateway spam, mitigating DDoS-like traffic patterns, and strictly minimizing resource consumption thereby reducing accumulated costs.

Frontend JavaScript Integration (script.js):

```javascript
// —— AWS Cloud Sync (Phase 4 Integration - Debounced) ——————————————
  // Clear the previous timer so it doesn't spam AWS while sliding
  clearTimeout(cloudSyncTimeout);

  // Set a new timer to wait 2 seconds after the user stops sliding
  cloudSyncTimeout = setTimeout(() => {
    const payload = {
      principal: principal,
      rate: rate,
      tenure: tenure,
      emi: emi,
      totalInterest: totalInterest,
      totalPayment: totalPayment
    };


fetch("https://96u69oekg4.execute-api.ap-south-1.amazonaws.com/dev/calculate
", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(payload)
    })
    .then(response => response.json())
    .then(data => console.log("AWS Sync Success:", data))
    .catch(error => console.error("AWS Sync Error:", error));
  }, 2000);
```

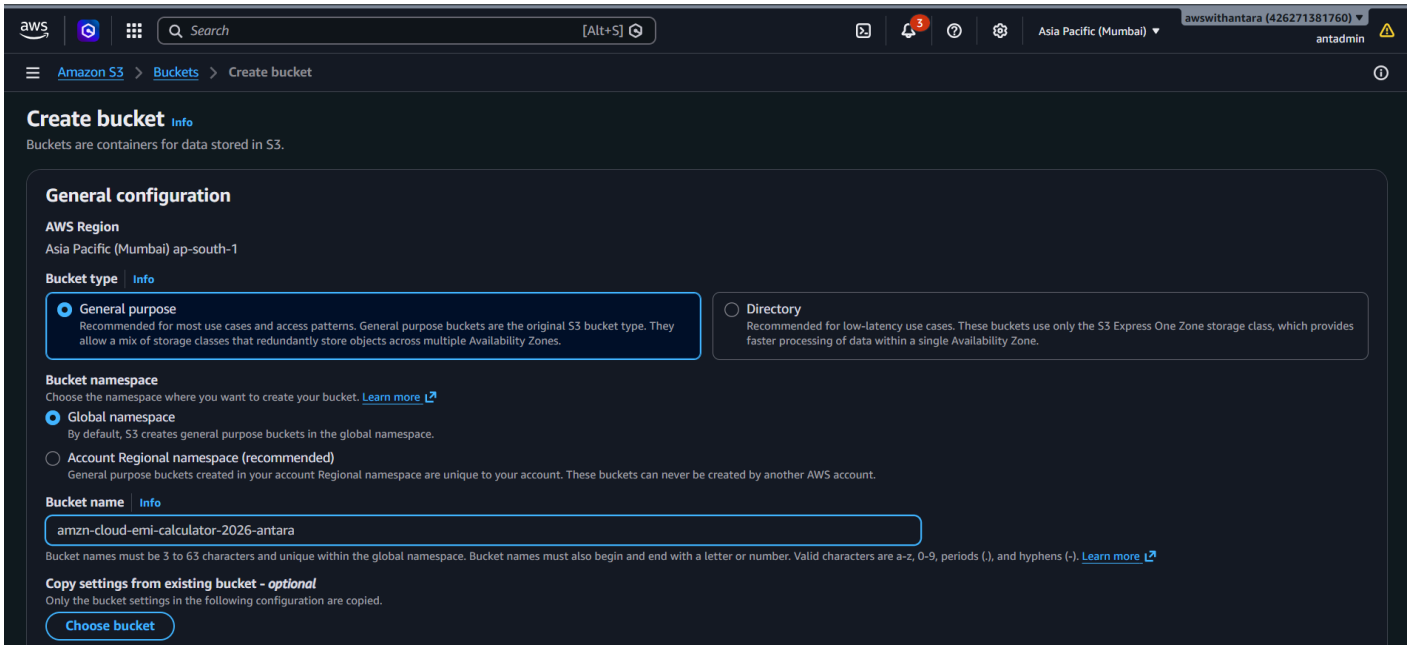
Phase 5: Amazon S3 Global Hosting

An S3 Bucket is provisioned comprising the Frontend (EMI Calculator Web Application) assets as objects, and configured explicitly for Static Website Hosting.

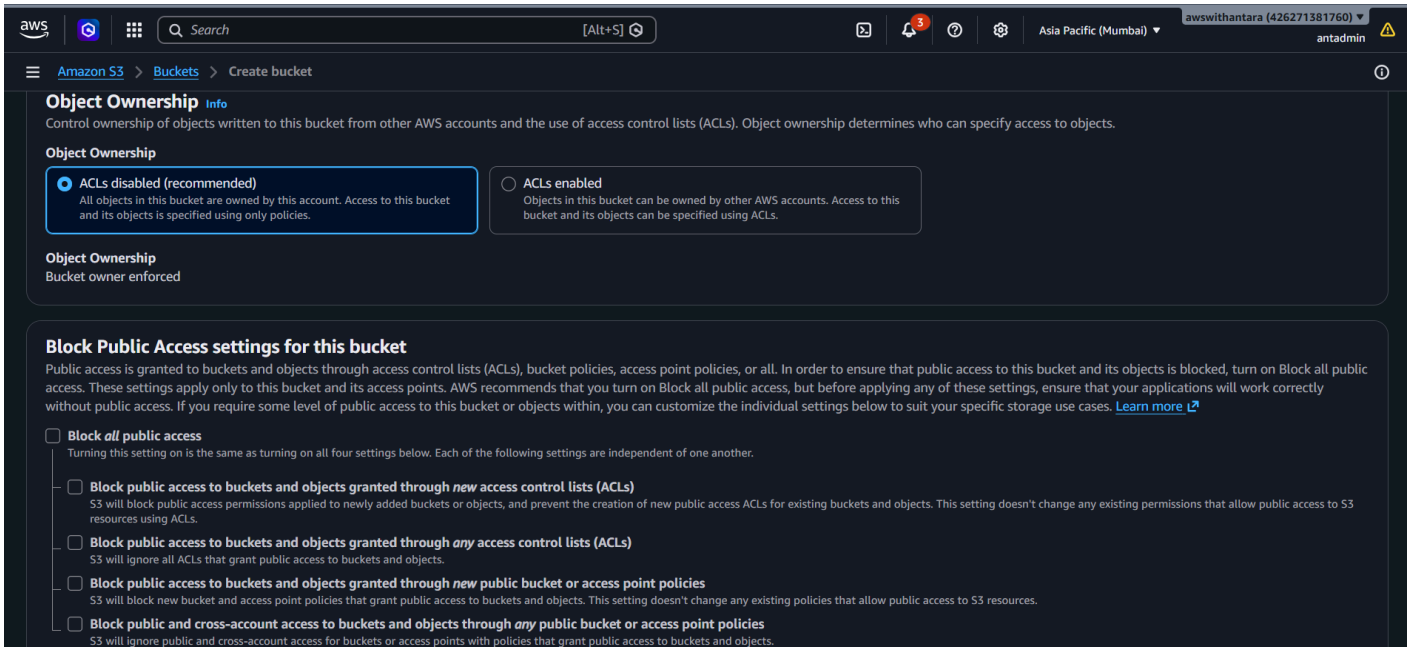
1) **S3 Bucket Details:**

- Name: amzn-cloud-emi-calculator-2026-antara
- Object Ownership: ACLs disabled (recommended)
- Block all public access: Must be disabled to allow public access (enabled by default)

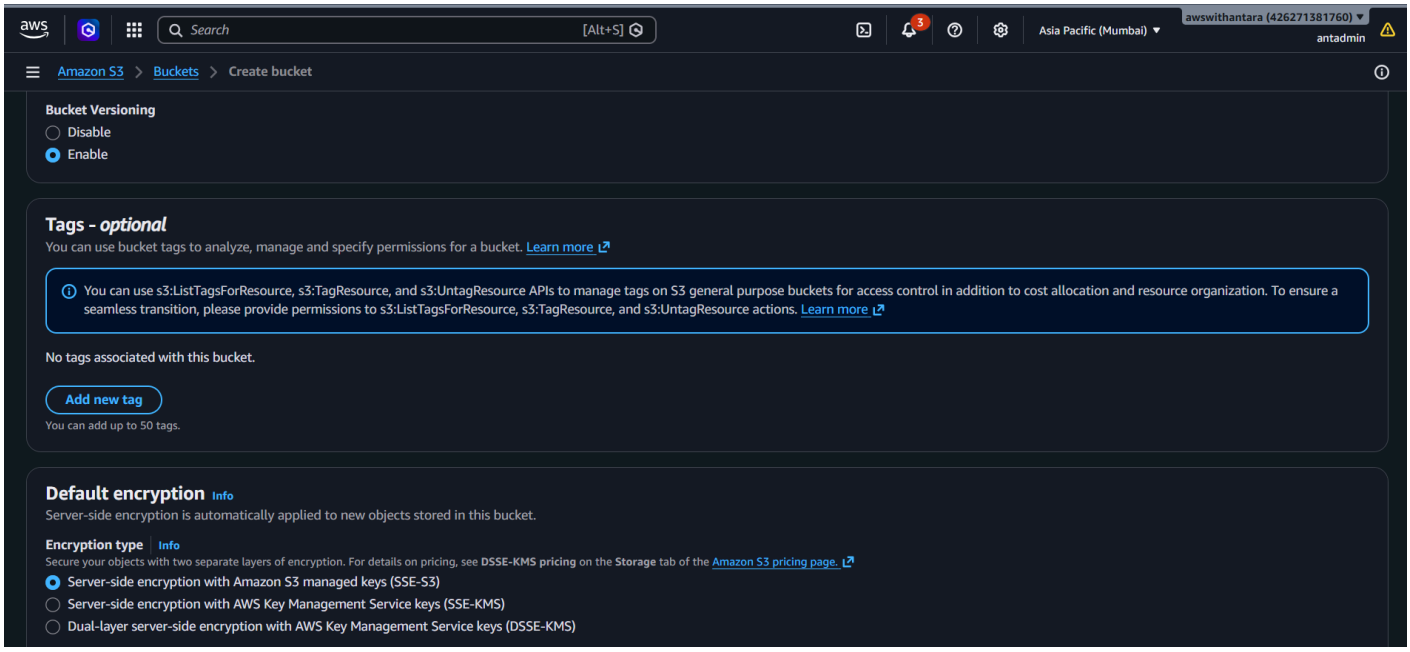
- Bucket Versioning: Must be enabled in order to prevent accidental deletions of data from the bucket. **Delete Markers** can be used with this feature to recover data in case of such deletions.



S3 Bucket Creation: Bucket type & name



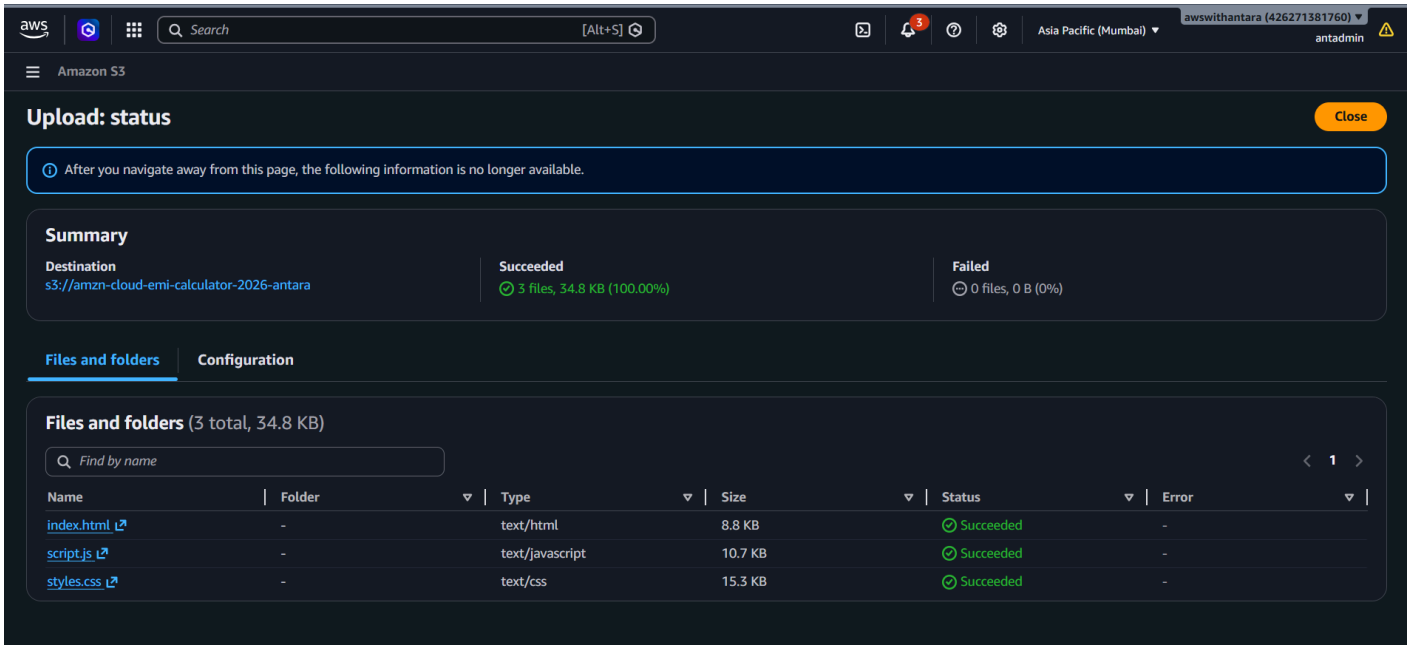
S3 Bucket Creation: Object ownership & Block all public access



S3 Bucket Creation: Bucket versioning & encryption

2) Objects Upload:

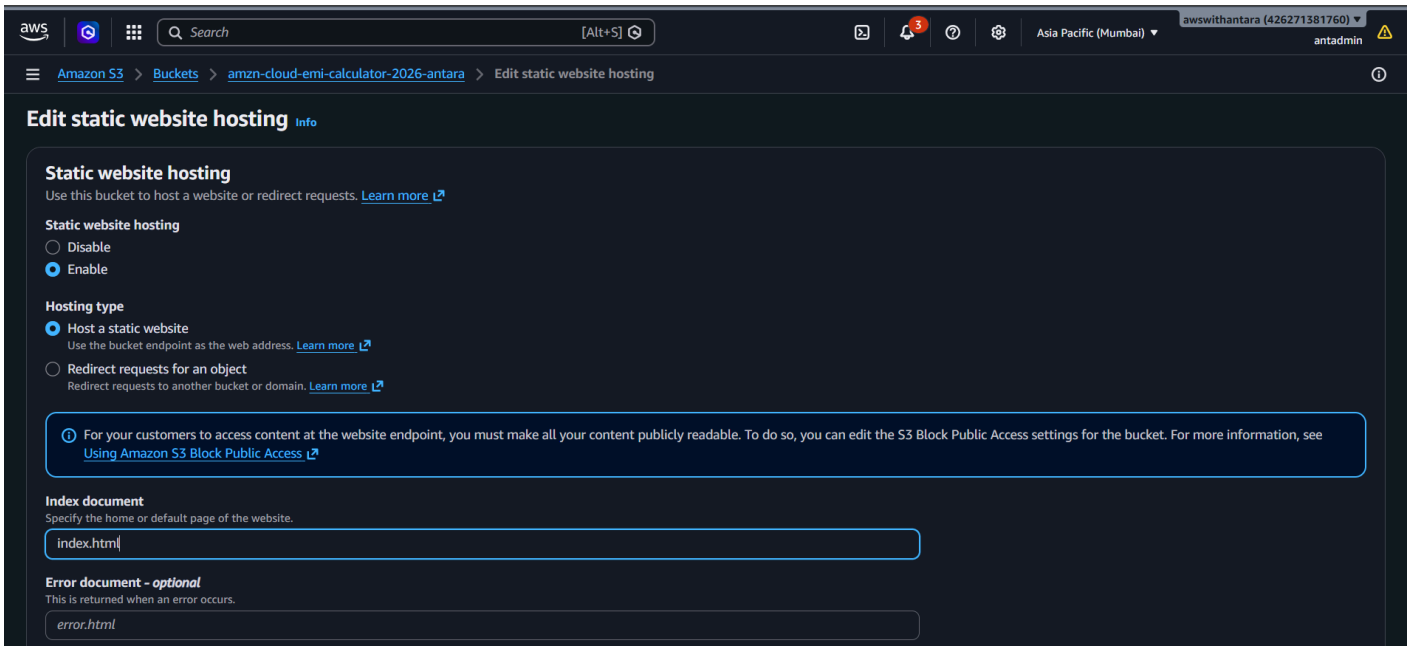
- The Frontend assets (**index.html**, **styles.css** & **scripts.js**) are uploaded in the newly created bucket **amzn-cloud-emi-calculator-2026-antara**



S3 Bucket: Objects upload status

3) Static Website Hosting:

- Hosting type: Host a static website
- Index document: index.html



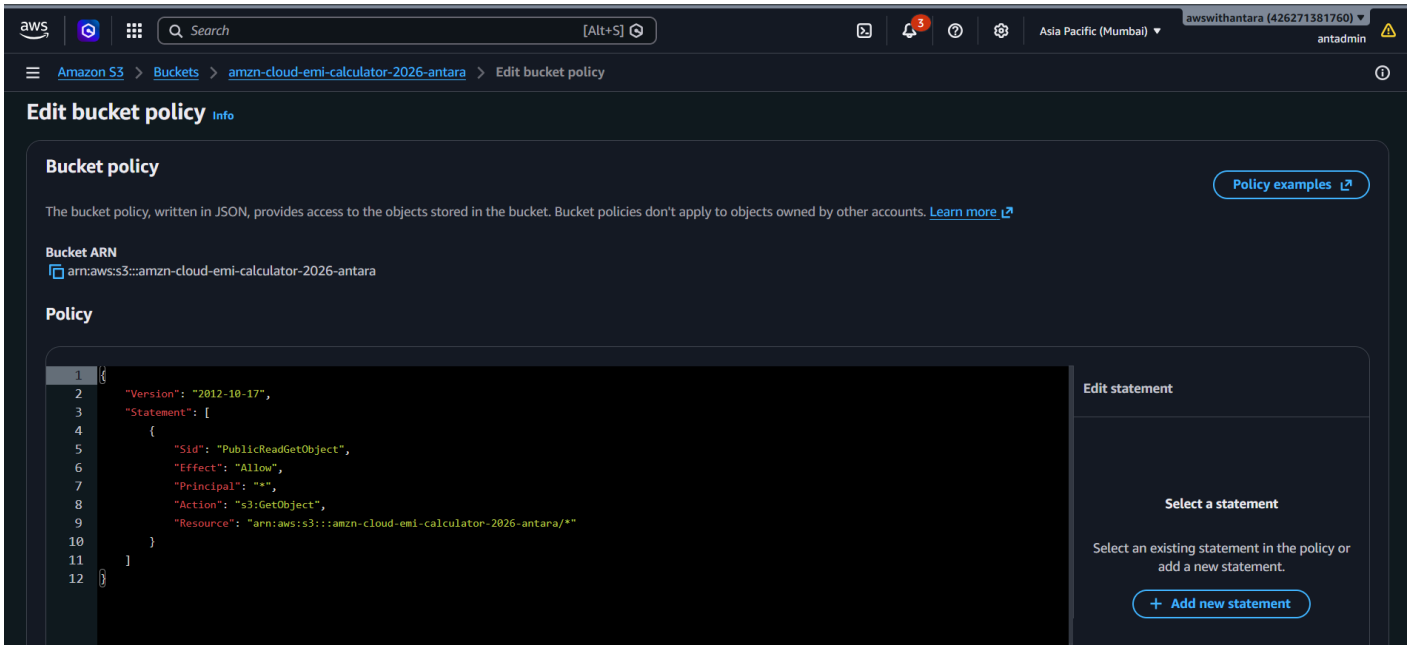
S3 Bucket: Static website hosting

4) Bucket Policy:

- In addition to disabling **Block all public access**, a bucket policy has to be defined in order to explicitly allow frontend access to any user on the internet.

JSON

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "PublicReadGetObject",
            "Effect": "Allow",
            "Principal": "*",
            "Action": "s3:GetObject",
            "Resource":
"arn:aws:s3:::amzn-cloud-emi-calculator-2026-antara/*"
        }
    ]
}
```



S3 Bucket: Bucket policy

Inference: This serverless architecture demonstrates a complete evolution from a static, client-side interface into a dynamic, cloud-native web application.

- By decoupling the architecture, this project achieves high availability and ultra-low latency on the frontend via Amazon S3 global edge hosting, while maintaining a secure, scalable, and stateful backend via API Gateway, AWS Lambda, and DynamoDB.
- Furthermore, the implementation of client-side request debouncing paired with On-Demand database capacity highlights a strong focus on cloud economics and resource optimization.